MUBD

Màster Universitari en Enginyeria de Dades Massives (Big Data)

Estadística

Índice

Métodos de clustering

01 Introducción

02 Clusterización jerárquica

- Distancia, inercia y criterio de Ward
- Algoritmo aglomerativo y partición

03 K-means

- Algoritmo, parámetros y convergencia
- Métodos y variantes

04 Medidas de rendimiento

05 Sistemas mixtos

Introducción

Objetivo

- Se desea formar grupos de observaciones similares entre sí y distintas entre grupos según unas características determinadas.
- Es un problema de clasificación de instancias.
- Hay distintas metodologías (algoritmos) de alcanzar este objetivo
  - Clusterización jerárquica
  - K-means
  - …

Introducción

Aplicabilidad

- Biología
  - Agrupar organismos en especies
  - Agrupar en familias genéticas
- Medicina
  - Análisis de imágenes
- Marketing
  - Crear segmentos de consumidores
  - Clasificación de productos
- Sociología
  - Organizar comunidades a través de relaciones en las redes sociales
  - Identificar grupos de estudiantes dentro de una comunidad educativa
- Otros…

Introducción

Tipos de problemas

- No supervisado (Clustering analysis)
  - Sin variable respuesta
  - Evaluación de la calidad del algoritmo complicada
  - Ej: clusterización jerárquica, K-means

- Supervisado (Discriminant analysis)
  - Se dispone de una variable respuesta
  - Fácil evaluación de la capacidad predictiva
  - Ej: conditional trees, random forest, KNN, Naive-Bayes, SVM

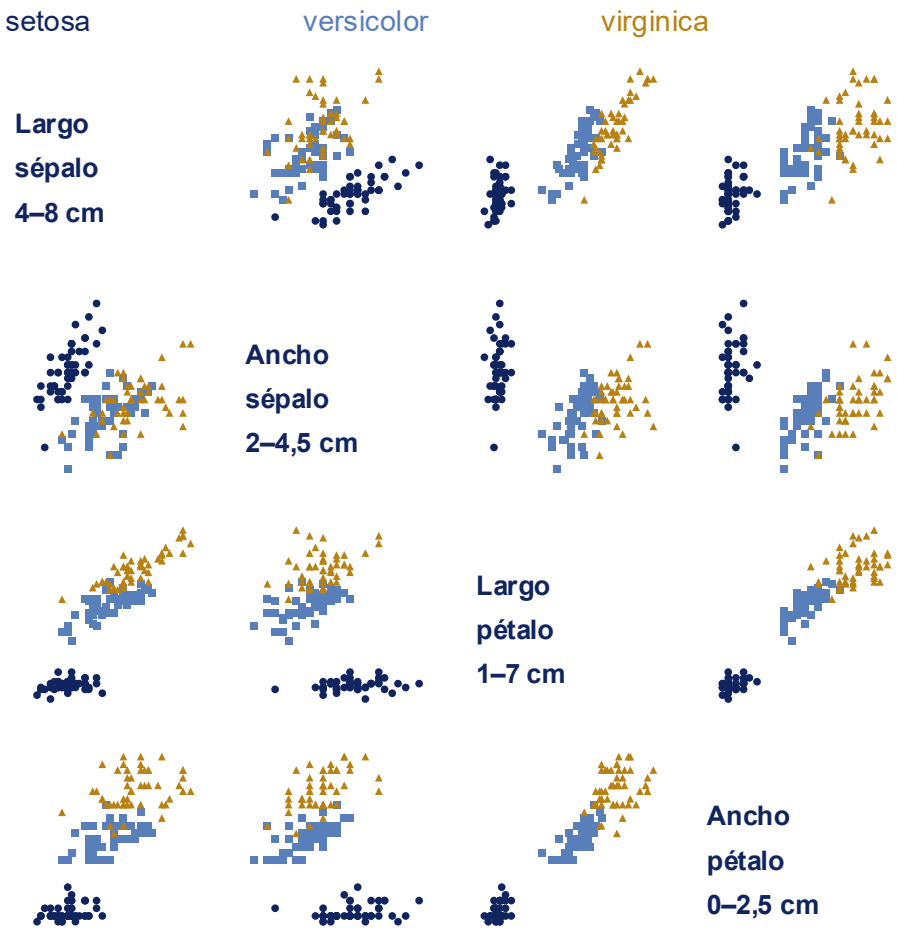
Datos del ejemplo

Iris: preparación común a los dos métodos

```
from sklearn.datasets import (
    load_iris)
from sklearn.preprocessing import (
    StandardScaler)
from pandas.plotting import (
    scatter_matrix)
import pandas as pd
import matplotlib.pyplot as plt

iris = load_iris()
X = StandardScaler().fit_transform(
    iris.data)
df = pd.DataFrame(iris.data,
    columns=iris.feature_names)
scatter_matrix(df, c=iris.target,
               figsize=(8, 6))
plt.show()
# 150 observaciones, 4 variables
# La especie no entra en X
```

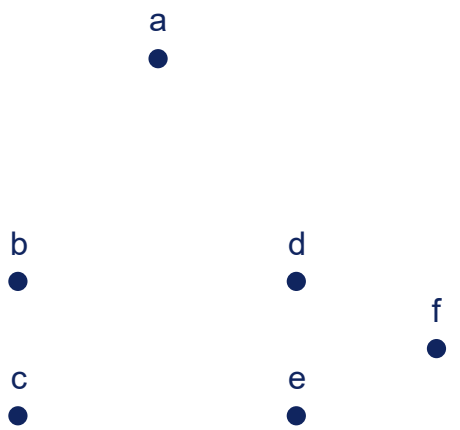
Matriz de dispersión de Iris (cm)



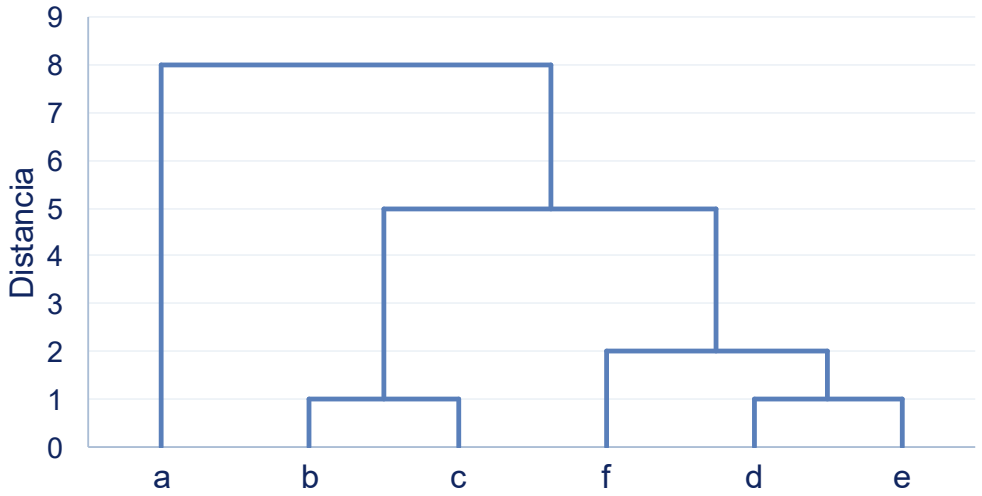
Clusterización jerárquica

Ejemplo

Observaciones (esquema)



Fusiones según la distancia completa



	A	B	C	D	E	F
A						
B	6					
C	7	1				
D	6	3	4			
E	7	4	3	1		
F	8	5	5	2	2	

	A	BC	DE	F
A				
BC	7			
DE	7	4		
F	8	5	2	

	A	BC	DEF
A			
BC	7		
DEF	8	5	

	A	BCDEF
A		
BCDEF	8	

Matrices de disimilitud del ejemplo original

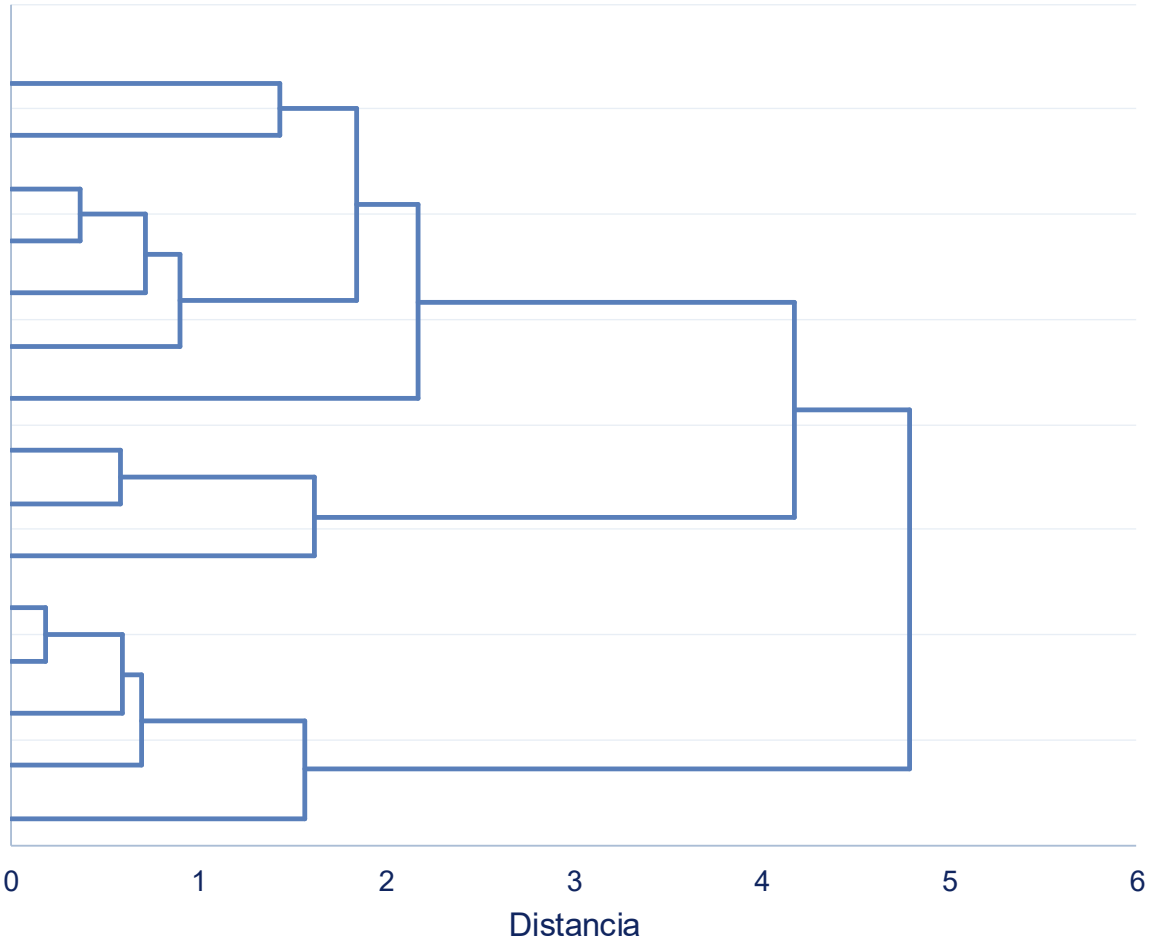
Clusterización jerárquica

Dendrograma

Un dendrograma muestra la secuencia de fusiones.

La distancia aumenta hacia la derecha.

Ejemplo ilustrativo: 15 observaciones de Iris y enlace completo.



Datos: Iris. Cuatro variables estandarizadas.

Clusterización jerárquica

Introducción

- Ventajas
  - No es imprescindible tener variables cuantitativas (aunque es más usual) ya que se pueden definir distancias para variables categóricas
  - No requiere definir el número de agrupaciones a priori. Se calcula un árbol jerárquico independientemente de los grupos
  - El árbol permite visualizar de forma intuitiva distancias entre individuos para tamaños muestrales no muy grandes
- Inconvenientes
  - Requiere definir un tipo de distancia
  - Requiere el cálculo de todos los pares de distancias. Mayor tiempo de computación para muestras grandes (>500)

Clusterización jerárquica

Proceso

Al realizar una clusterización jerarárquica, se deben realizar los siguientes pasos (en azul, decisiones a tomar):

1. Decidir el tipo de distancia a emplear (euclídea, manhattan…)
2. Construir la matriz de disimilitudes
3. Decidir el método de agrupación (aglomerativo o divisivo)
4. Decidir el sub-método de agrupación (Ward, completo…)
5. Aplicar el algoritmo de clusterización
6. Decidir el criterio para realizar la partición de los clústeres
7. Particionar la muestra

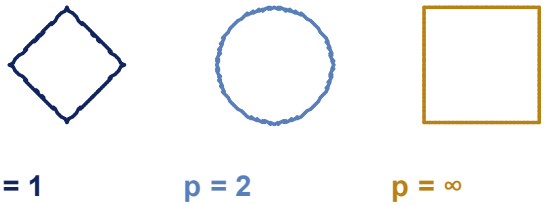
NOTA: Con los pasos 1 a 5 construimos el dendograma

Distancias entre observaciones

Tipos

- ***Minkowski*** $d(i,l) = \left(\sum_{k=1}^{K} |x_{ik} - x_{lk}|^p\right)^{\frac{1}{p}}$
  - ***Manhattan*** *(p = 1)* $d(i,l) = \sum_{k=1}^{K} |x_{ik} - x_{lk}|$
  - ***Euclídea*** *(p = 2)* $d^2(i,l) = \sum_{k=1}^{K} (x_{ik} - x_{lk})^2$
  - ***Máxima*** *(p = ∞)* $d(i,l) = \max_k \|x_{ik} - x_{lk}\|$

Contornos de distancia unidad



p = 1 p = 2 p = ∞

- ***Mahalanobis*** $d^2(i,l) = (x_i - x_l)^{\mathrm{T}} \Sigma^{-1} (x_i - x_l)$ → Σ es la matriz de var-covar

- ***Canberra*** $d(i,l) = \sum_{k=1}^{K} \frac{|x_{ik} - x_{lk}|}{|x_{ik} + x_{lk}|}$

- ***Gower*** $d_k(i,l) = \sum_{k=1}^{n} d_k(i,l) \rightarrow d_k(i,l) = \begin{cases} 1 \; si \; i_k \neq j_k \\ 0 \; si \; i_k = j_k \end{cases}$ (variables categóricas)

- ***Índice de Jaccard*** $d(i,l) = \frac{\boldsymbol{n_{++}}}{\boldsymbol{n_{++}} + \boldsymbol{n_{+-}} + \boldsymbol{n_{-+}}}$ (variables categóricas)

Distancias entre grupos de observaciones

Tipos

- ***Simple.*** Mínima distancia entre dos puntos pertenecientes uno a cada grupo
- ***Completa.*** Máxima distancia entre dos puntos pertenecientes uno a cada grupo
- ***Entre centros de gravedad.*** Distancia entre los centros de gravedad
- ***Criterio de Ward.*** No es una distancia en sí, sino un criterio para hacer la agrupación que consiste en minimizar la inercia.

Inercia

Definición

- La inercia es una medida de la heterogeneidad existente en un conjunto de datos.
- Dada una partición en Q grupos formados por I_q elementos en un espacio K-dimensional (K variables) se define:

Inercia Total	$\sum_{k=1}^{K}\sum_{q=1}^{Q}\sum_{i=1}^{I_q}(y_{iqk}-\bar{y}_k)^2$	*[Distancia de cada punto al centro de gravedad]*
Inercia Entre-grupo	$\sum_{k=1}^{K}\sum_{q=1}^{Q} I_q(\bar{y}_{qk}-\bar{y}_k)^2$	*[Distancia del centro de los grupos al centro global]*
Inercia Intra-grupo	$\sum_{k=1}^{K}\sum_{q=1}^{Q}\sum_{i=1}^{I_q}(y_{iqk}-\bar{y}_{qk})^2$	*[Distancia de cada punto al centro de su grupo]*

- Se cumple que:

Inercia Total = Inercia Entre-grupo + Inercia Intra-grupo

Inercia

Propiedades

- La calidad de una partición puede medirse como el % de variabilidad explicada por los grupos (similar al R^2 del modelo lineal), es decir:

$$Variabilidad\ explicada = \frac{Inercia\ entre - grupos}{Inercia\ total}$$

- El porcentaje de variabilidad explicada siempre crece a medida que se incrementan el número de grupos:

Mayor número de grupos → **Más** variabilidad explicada → **Más** inercia entre-grupos

Menor número de grupos → **Menos** variabilidad explicada → **Menos** inercia entre-grupos

Método de agrupación

Algoritmo aglomerativo y alternativa divisiva

AGNES: cada observación empieza como un clúster.

1. Construir la matriz de distancias.

2. Unir los dos grupos más próximos según el enlace elegido, por ejemplo Ward.

3. Representar la fusión a la altura de su distancia o cambio de inercia.

4. Actualizar las distancias al nuevo grupo y repetir hasta obtener una raíz.

Alternativa divisiva (DIANA)

Empieza con todas las observaciones en un grupo y lo divide progresivamente. La dirección del proceso es la inversa de la aglomeración.

Criterio de Ward

Agrupación

- Se parte de *Q* clústeres
- Se desea una agrupación ideal que pase a *Q-1* clústeres
- Sea el clúster *p* (con centro de gravedad g_p y tamaño I_p) y el clúster *q* (con centro de gravedad g_q y tamaño I_q). El incremento en la inercia intra-grupo se cuantifica por:

$$\Delta(p,q) = \frac{I_p I_q}{I_p + I_q} d^2(g_p, g_q)$$

- El criterio de Ward consiste en juntar aquellos clústeres tales que minimicen el incremento en la inercia intra-grupo al pasar de *Q* a (*Q-1*) clústeres.
- Este criterio favorece juntar:
  - Los clústeres cercanos
  - Los clústeres pequeños

Calidad del árbol jerárquico

Distancia y correlación cofenética

- Para evaluar la calidad del árbol (dendograma) construido se deben comparar cuán similares son las distancias (alturas) producidas por el dendograma respecto a las distancias originales.
- Las distancias obtenidas del dendograma se denominan **distancias cofenéticas**.
- Para evaluar la calidad, se calcula la **correlación cofenética** entre estas distancias y las distancias originales.
- Si la agrupación es válida, debe existir una fuerte correlación entre estas distancias. Se considera que los valores superiores a 0.75 son aceptables.
- Escoger distancias “medias” para la agrupación produce valores altos de esta correlación.

Clusterización jerárquica

Número de clústeres

- Una vez se tiene el dendograma, se debe definir una partición en *K* clústeres.
- Criterios para escoger una partición:
  - **Visual**. A partir del dendograma
  - **Cambio de inercia**
    - Regla del codo.
    - Minimizar el cociente Δ(q) / Δ(q+1) donde Δ(q) es el cambio en la inercia intra al pasar de *q* a *q-1* clústeres
  - **Parsimonia**. No es conveniente escoger un gran número de clústeres
  - **Interpretabilidad**. Los grupos deben tener algún sentido

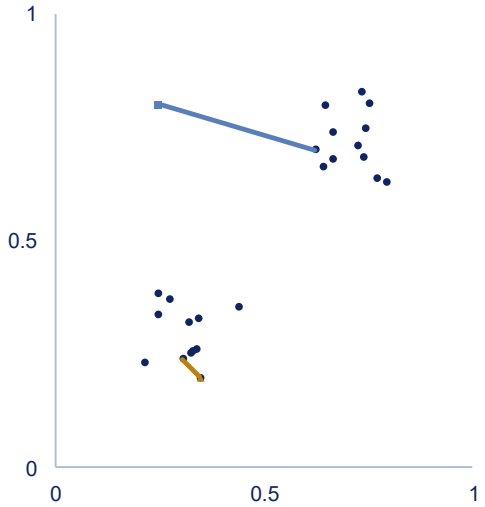
Tendencia a la agrupación

Estadístico de Hopkins

- Escoger n puntos aleatorios D'=($p_1,\ldots,p_n$) de nuestro conjunto de datos (D)
- Para cada punto $p_i \in D'$, buscar su vecino más próximo $p_j \in D$ y calcular su distancia (w_i)
- Generar un conjunto de datos simulado (R) con distribución uniforme con n puntos ($q_1,\ldots,q_n$) y la misma variabilidad que el conjunto de datos D.
- Para cada punto $q_i \in D$, buscar su vecino más próximo dentro del conjunto real de puntos y calcular su distancia (u_i)
- El estadístico de *Hopkins* será:

$$H = \frac{\sum u_i}{\sum w_i + \sum u_i}$$

- Un valor de 0.5 indicará que los clústeres son parecidos y que por tanto, no existen clústeres. Un valor próximo a 1 indicará presencia de clústeres.
- **Nota: en Python, definir explícitamente la convención de H antes de comparar implementaciones.**



Ejemplo simulado

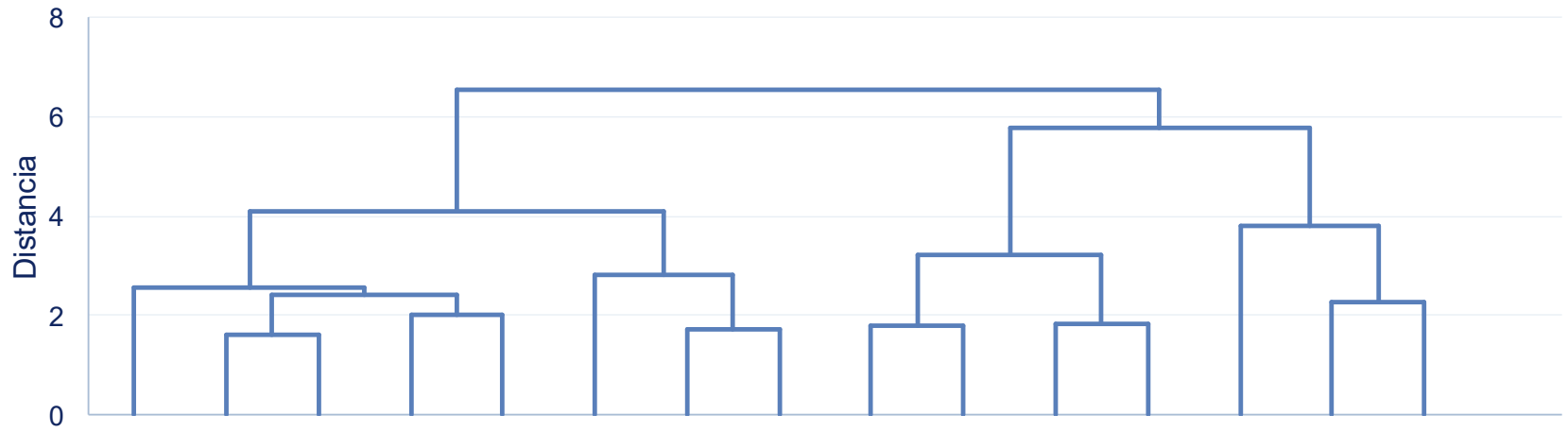
u: punto uniforme al dato más próximo

w: dato al vecino más próximo

Clusterización jerárquica

Python: construir el árbol y obtener los grupos

```
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
import matplotlib.pyplot as plt
import numpy as np
Z = linkage(X, method="complete")
dendrogram(Z, truncate_mode="lastp", p=15)
plt.show()
lab_h = fcluster(Z, 3, criterion="maxclust")
print(np.bincount(lab_h)[1:])  # [77 24 49]
```



Árbol sin k previo. Corte en un máximo de 3 grupos. Las etiquetas son arbitrarias.

K-means

Introducción

- Tipos de variables: idealmente se precisan variables cuantitativas o ordinales con un gran número de categorías (aunque pueden implementarse distancias para otro tipo de variables)
- Ventajas
  - No requiere del cálculo de todos los pares de distancias
  - Requiere menos tiempo de computación que la clusterización jerárquica
- Inconvenientes
  - Resultados inaceptables para según qué tipo de clústeres (sobre todo NO convexos). Sol: usar versiones de K-means que minimicen este problema.
  - Distintos resultados dependiendo de los parámetros iniciales (sobre todo para conjuntos de datos pequeños). Sol: fijar distintos puntos iniciales.
  - Requiere fijar el número de clústeres a priori. Sol: si el coste computacional no es muy alto, probar diversos números de clústeres.
  - Al basarse en distancias medias, los *outliers* pueden afectar drásticamente al resultado. Sol: probar algoritmos más robustos (K-medians, K-mediods).

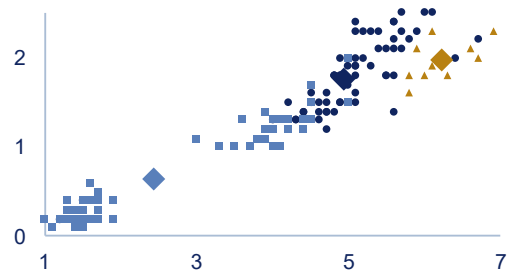
K-means

Algoritmo y evolución de los grupos con Iris

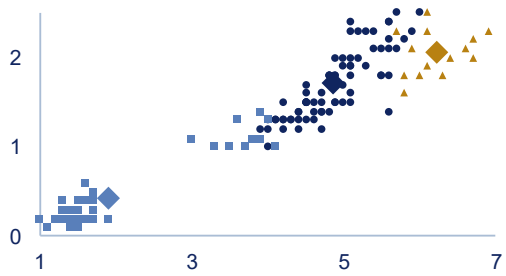
Objetivo: minimizar la inercia interna.

Inicio: fijar k y seleccionar los centros iniciales (aquí, k = 3).

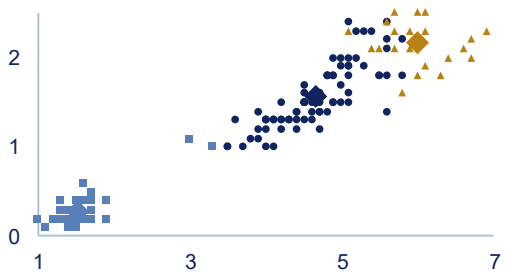
Iteración 1



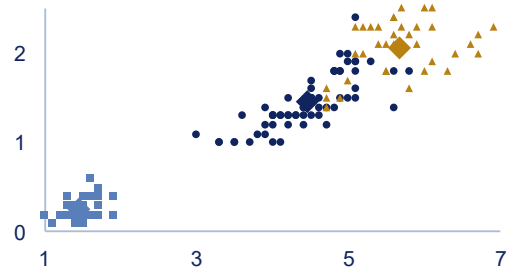
Iteración 2



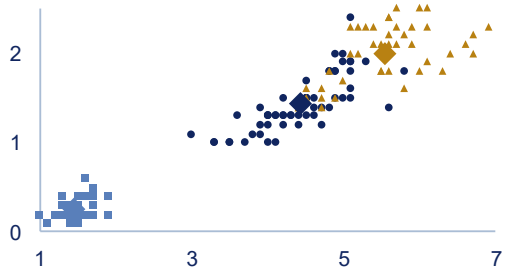
Iteración 3



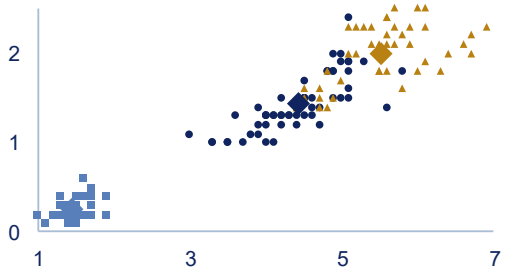
Iteración 5



Iteración 7



Convergencia



En cada iteración: asignar al centro más próximo y recalcular los centros.

Parada: cambio inferior al umbral o máximo de iteraciones.

Proyección en 2 variables de un ajuste con 4. X: largo del pétalo. Y: ancho (cm). Centros: marcadores grandes.

K-means

Python: parámetros y convergencia

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, init="k-means++",
            n_init=10, max_iter=300,
            algorithm="lloyd", random_state=42)
km.fit(X)

# n_clusters: número de grupos
# n_init: ejecuciones con distintos centros iniciales
# max_iter: máximo de iteraciones por ejecución
# random_state: semilla reproducible
# labels_: grupos; cluster_centers_: centros
# inertia_: suma de distancias al cuadrado
```

n_init conserva el ajuste con menor inercia entre las ejecuciones.

La inercia no aumenta al iterar, pero la convergencia no garantiza el óptimo global. Comprobar que max_iter permite converger.

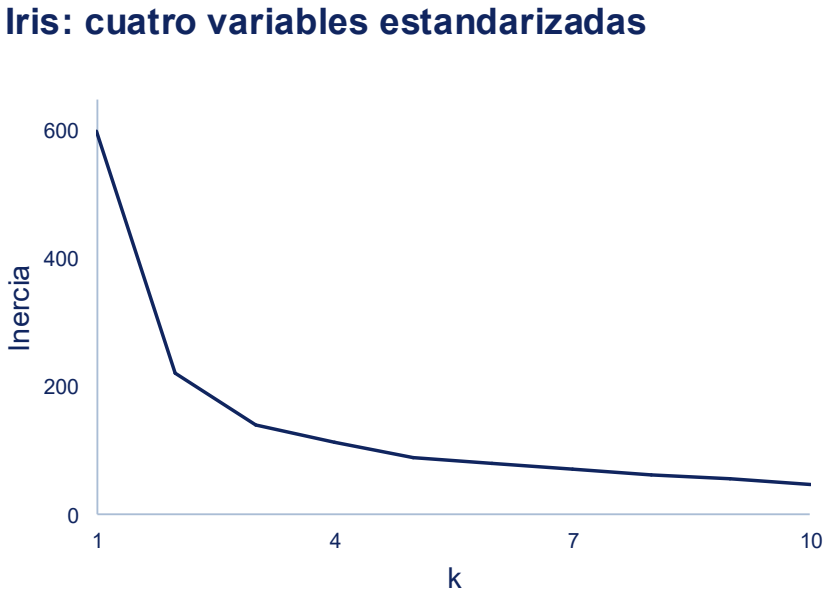
K-means

Número de grupos: el codo aplicado a Iris

La inercia interna disminuye al aumentar k.

El codo señala dónde la mejora empieza a ser menor.

Orienta la elección, pero no determina por sí solo el número de grupos.



Variabilidad entre grupos: 62,9 % (k = 2), 76,7 % (k = 3), 81,0 % (k = 4).

La mejora de 3 a 4 grupos es menor que la de 2 a 3.

Otros indicadores e interpretabilidad complementan el codo.

K-means

Métodos

En Python, sklearn.cluster.KMeans permite elegir:

algorithm="lloyd": alterna asignación al centro más cercano y actualización de los centros.

algorithm="elkan": usa cotas basadas en la desigualdad triangular para evitar algunos cálculos de distancia. Puede requerir más memoria.

init="k-means++" e init="random" son inicializaciones, no métodos de actualización.

Hartigan–Wong y MacQueen no son opciones de algorithm en esta clase.

K-means

Inicialización, eficiencia y métodos alternativos

Categoría	Opciones y propósito
Inicialización	K-means++ elige los centros iniciales.
Eficiencia	MiniBatchKMeans usa lotes. KD-trees puede acelerar búsquedas.
Otros centros	K-medians usa medianas. K-medoids usa observaciones representativas.
Otros enfoques	Fuzzy C-means usa grados difusos. GaussianMixture (EM) usa probabilidades. DBSCAN agrupa por densidad.
Geometrías especiales	Spherical k-means: datos direccionales. Minkowski ponderado: pesos por variable.

Estas opciones tienen funciones distintas y no son variantes equivalentes de K-means. Para datos mixtos o longitudinales se requieren métodos y distancias adecuados.

Variantes

K-mediods

- La diferencia básica con el K-means es que en vez de escoger el centro de gravedad como centro del clúster, se escoge el **elemento (*mediod*) más representativo**
- No confundir con el algoritmo K-medians que se basa en las medianas univariantes.
- El *mediod* es aquel punto cuya **distancia media al resto de puntos del clúster es mínima**
- Es un algoritmo **más robusto** que el K-means ya que no es tan sensible a la presencia de outliers
- Una variante del algoritmo de **K-mediods** es **CLARA** (*CLustering LARge Applications*) que mejora el rendimiento para grandes volúmenes de datos
- En Python, KMeans de scikit-learn no implementa K-medoids ni CLARA. Requieren otra implementación.

Medidas de rendimiento

Tipos

Medidas internas: evalúan la agrupación sin una referencia externa.

Ejemplos simulados

Compacidad



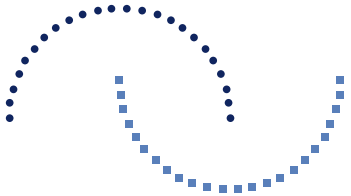
Baja variabilidad dentro de cada grupo.

Separación



Alta variabilidad entre grupos.

Conectividad



Elementos conexos en el mismo clúster.

Medidas externas: comparan con otra agrupación o con un conjunto previamente etiquetado.

Clustering

Medidas internas: una agrupación

- ***Inercia intra*** (suma de las distancias euclídeas al cuadrado de cada punto a su respectivo centroide). Valores entre 0 y ∞. Decrece monótonamente al aumentar el número de clústeres:

$$I = \sum_{i=1}^{k} \sum_{j=1}^{n_k} (x_{ij} - c_j)^2$$

- ***Dunn index*** (cociente entre distancia mínima entre puntos de distintos clústeres y la distancia máxima entre puntos del mismo clúster). Valores entre 0 y ∞. Se debe maximizar. *Python: calcular a partir de la matriz de distancias.*

$$\mathrm{D} = \frac{d_{mín}}{d_{máx}}$$

- ***Silhouette coefficient***. Valores entre -1 y 1. Se debe maximizar. Para un punto concreto *i*, a(i) es la media de las distancias a los puntos del mismo clúster y b(i), la mínima distancia a puntos de otro clúster. Este coeficiente es la media de todos los s(i). Python: sklearn.metrics.silhouette_score(X, labels)

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

Clustering

Medidas externas: comparación de agrupaciones

- Para cada pareja de puntos en 2 agrupaciones (C1 y C2), se define:

A: parejas en mismo clúster en ambas agrupaciones C1 y C2
B: parejas en distintos clústeres en ambas agrupaciones C1 y C2
C: parejas en mismo clúster en C1 y distinto clúster en C2
D: parejas en distinto clúster en C1 y mismo clúster en C2

- ***Rand index*** (Proporción de parejas igual de agrupadas en ambas agrupaciones). Toma valores entre 0 y 1. Cuánto mayor, más similitud. Python: sklearn.metrics.rand_score(C1, C2)

$$R = \frac{\#\ coincidencias\ por\ parejas}{\#\ parejas} = \frac{A + B}{A + B + C + D}$$

- ***Jaccard index*** (Proporción de parejas en el mismo clúster). Toma valores entre 0 y 1. Cuánto mayor, más similitud. Python: A / (A + C + D), sobre pares de observaciones.

$$J = \frac{\#\ parejas\ en\ el\ mismo\ clúster}{\#\ parejas\ -\ \#parejas\ en\ distintos\ clústeres\ en\ ambas} = \frac{A}{A + C + D}$$

Sistemas mixtos

K-means + Clusterización jerárquica

- **Opción 1**: Útil con un gran número de observaciones
  - **Etapa 1**. Se realiza un *k-means* con un elevado número de grupos (p.ej, 100)
  - **Etapa 2**. Se realiza una clusterización jerárquica de los grupos usando sus centros de gravedad.

- **Opción 2**: Útil con pocas observaciones. En ciertos casos, mejora el rendimiento
  - **Etapa 1**. Se realiza una clusterización jerárquica
  - **Etapa 2**. Partiendo de una partición inicial de esta clusterización se pueden reasignar los puntos según el algoritmo de k-means

Sistemas mixtos

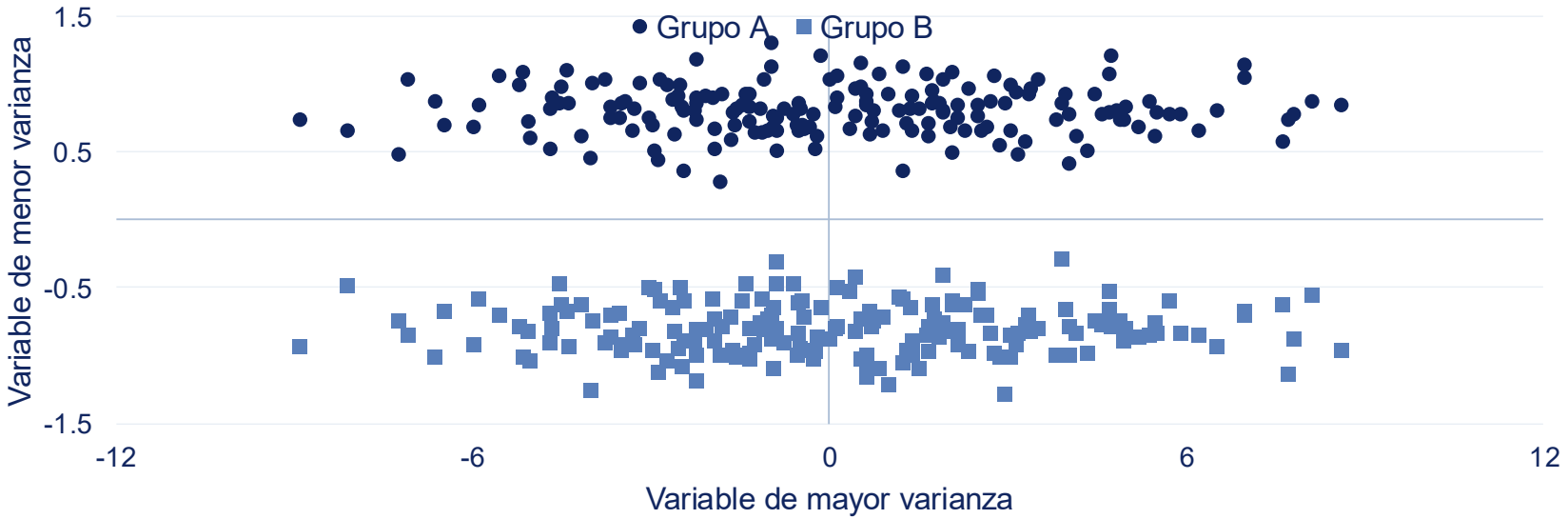
ACP o ACM con clustering

El ACP o ACM puede reducir ruido y representar variables de distintos tipos.

Se retienen las componentes con varianza no nula.

Después se aplica clustering con distancia euclídea.

En algunos casos, el ACP puede ser contraproducente.



Ejemplo simulado: la mayor varianza puede ocultar la separación entre grupos.

K-means

Python: comparación de 2, 3 y 4 grupos

```
from sklearn.cluster import KMeans
import numpy as np
models = {}
for k in (2, 3, 4):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    models[k] = km
    print(k, np.bincount(km.labels_), round(km.inertia_, 3))
km3 = models[3]  # Modelo usado en la comparación externa
```

k	Tamaños por etiqueta	Inercia W	Entre grupos
2	100 / 50	222,362	62,9 %
3	53 / 50 / 47	139,820	76,7 %
4	47 / 53 / 28 / 22	114,093	81,0 %

Mismos datos estandarizados y semilla 42. Las etiquetas son arbitrarias.
Centros estandarizados: models[k].cluster_centers_. La especie queda fuera.

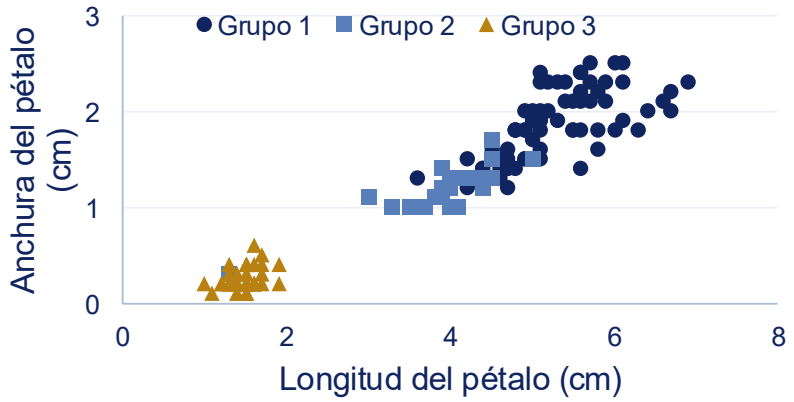
Comparación de los dos métodos

Iris: tres grupos y una misma referencia externa

ARI mide acuerdo ajustado por azar, no acierto. La especie queda fuera del ajuste.
Las etiquetas son arbitrarias. El modelo usa 4 variables y los gráficos muestran 2.

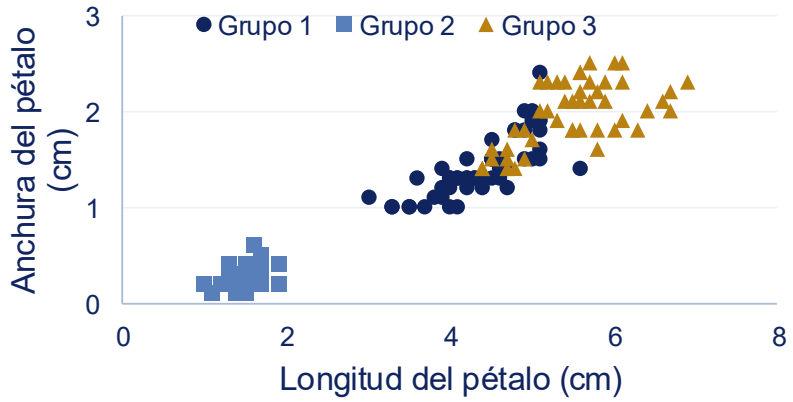
Jerárquico (completo): ARI = 0,573

	Clúster		
Especie	1	2	3
setosa	0	1	49
versicolor	29	21	0
virginica	48	2	0



K-means: ARI = 0,620

	Clúster		
Especie	1	2	3
setosa	0	50	0
versicolor	39	0	11
virginica	14	0	36




```
from sklearn.metrics import adjusted_rand_score
for labels in (lab_h, km3.labels_):
    print(adjusted_rand_score(iris.target, labels))
```

MUBD

Màster Universitari en Enginyeria de Dades Massives (Big Data)

Estadística